

Диаграммы, схемы, графы

Применение средств визуализации
для документирования моделей

Назначение UML / Unified Modeling Language

Моделирование = *(контекстно)*

- ✓ Спецификация
 - ❖ формальная и наглядная нотация
- ✓ Визуализация
 - ❖ для общения (исполнители, заказчики, stakeholders)
- ✓ Проектирование
 - ❖ архитектура /конструирование / семантика
- ✓ Документирование
 - ❖ по разным аспектам / прагматика

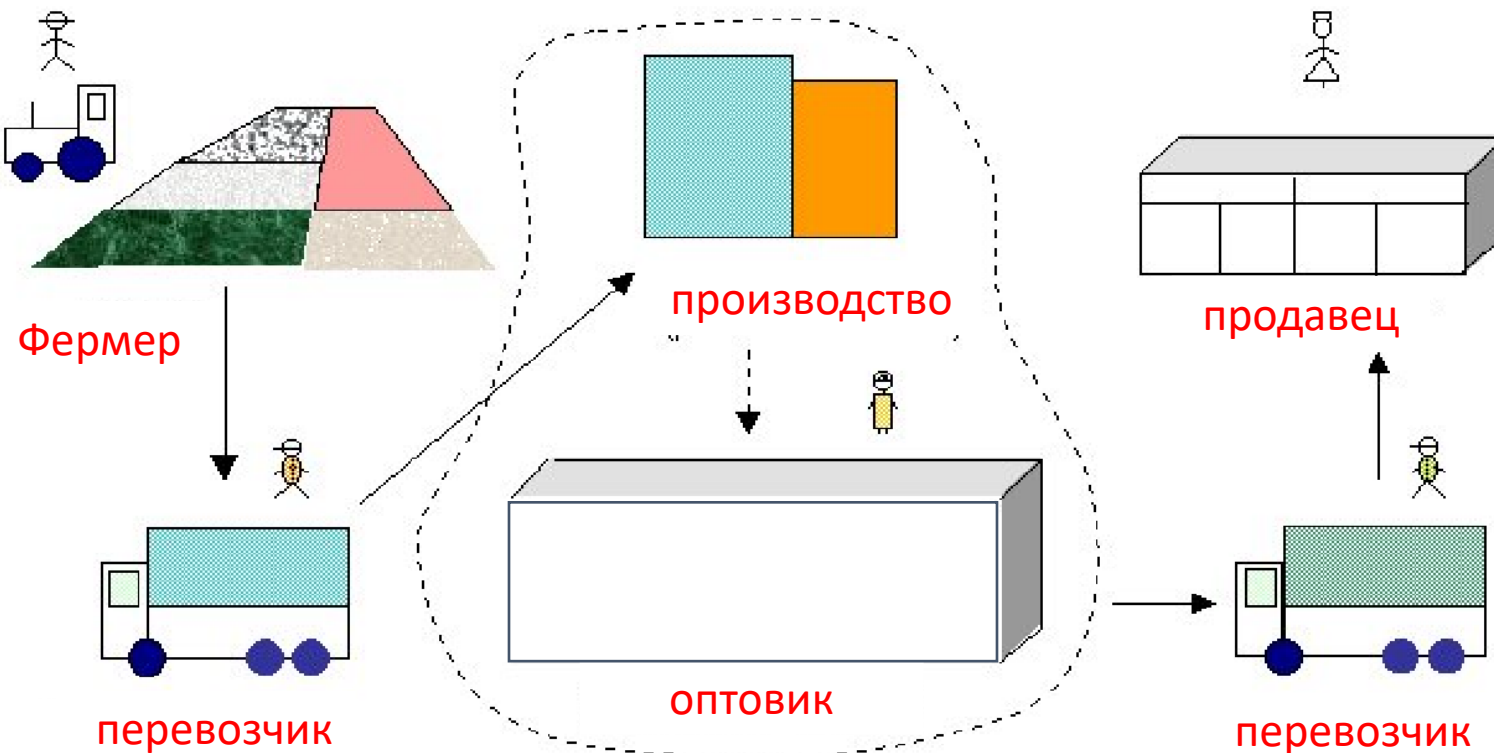


Визуализация

Сухой текст < текст с картинками < картинки с текстом

=> **UML**

➔ *КОМИКСЫ*



Назначение UML / Unified Modeling Language

нотация UML предлагает пять представлений системы

Аспект прецедентов

Аспект проектирования

Аспект процессов

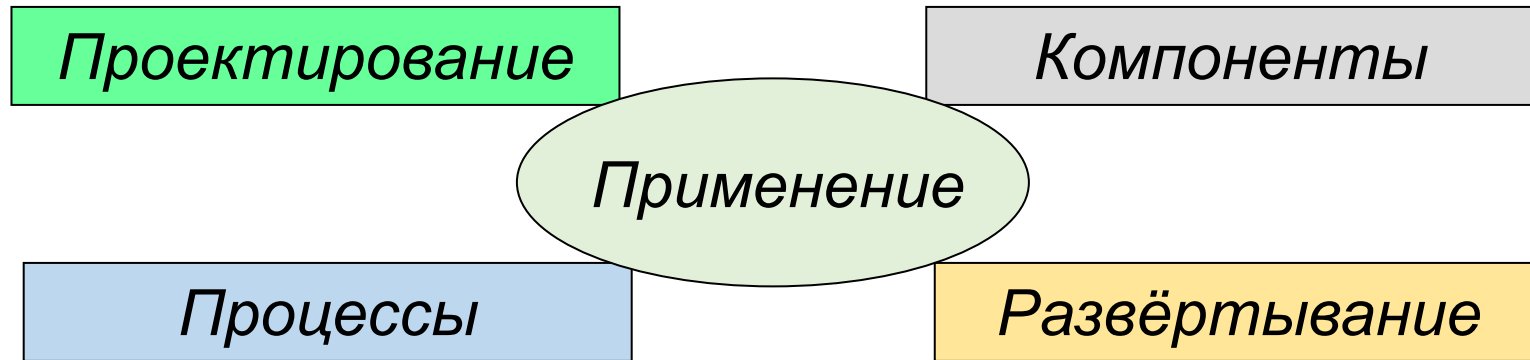
Аспект развертывания

Аспект реализации

Представление жизненного цикла ИС

словарь предметной
области, функциональность

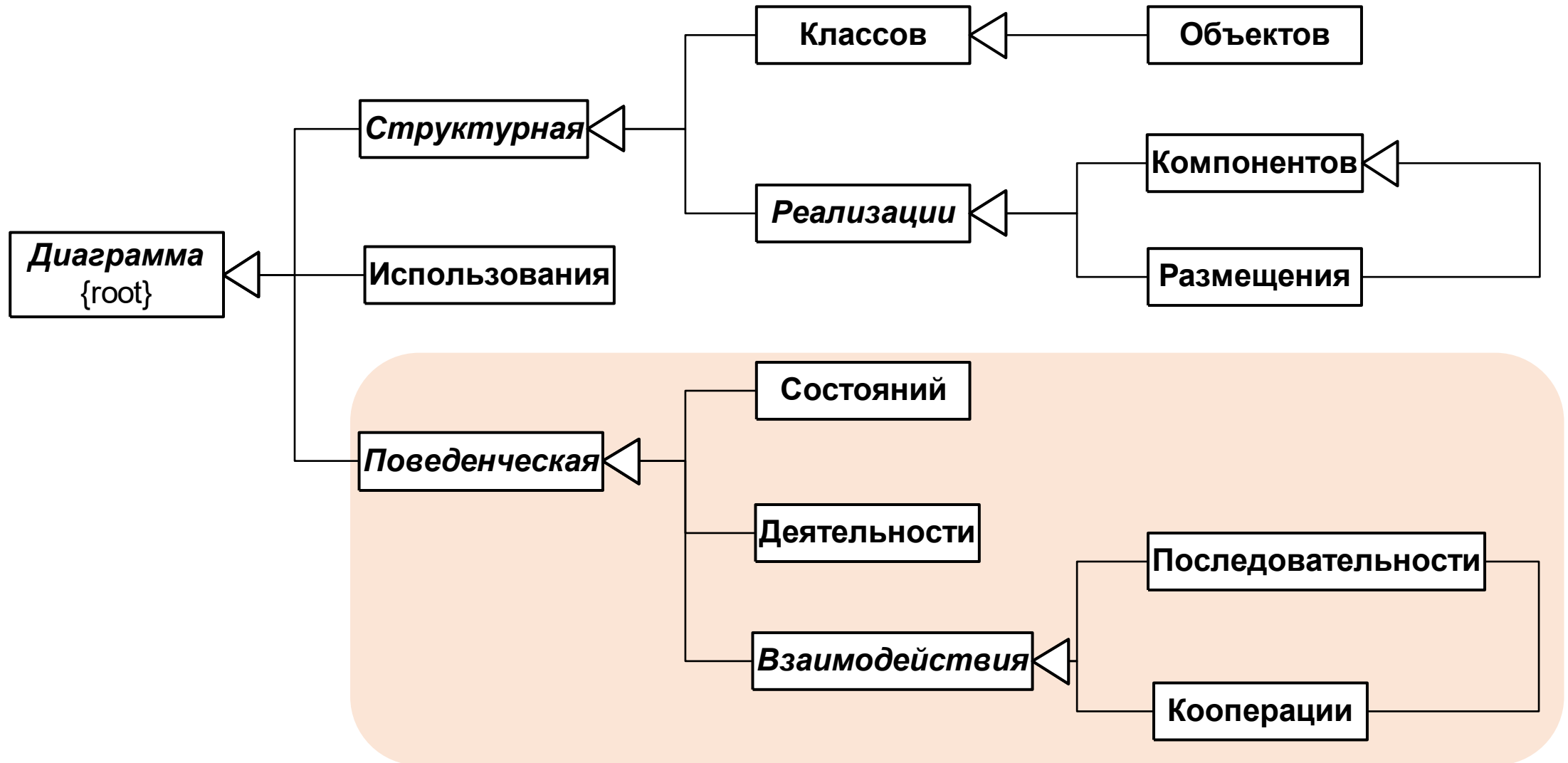
сборка системы, управление
конфигурацией



деятельность,
масштабируемость,
пропускная способность

распределение,
установка,
топология системы

Типы диаграмм UML



Представление поведения

Как работает система ?

Диаграммы -

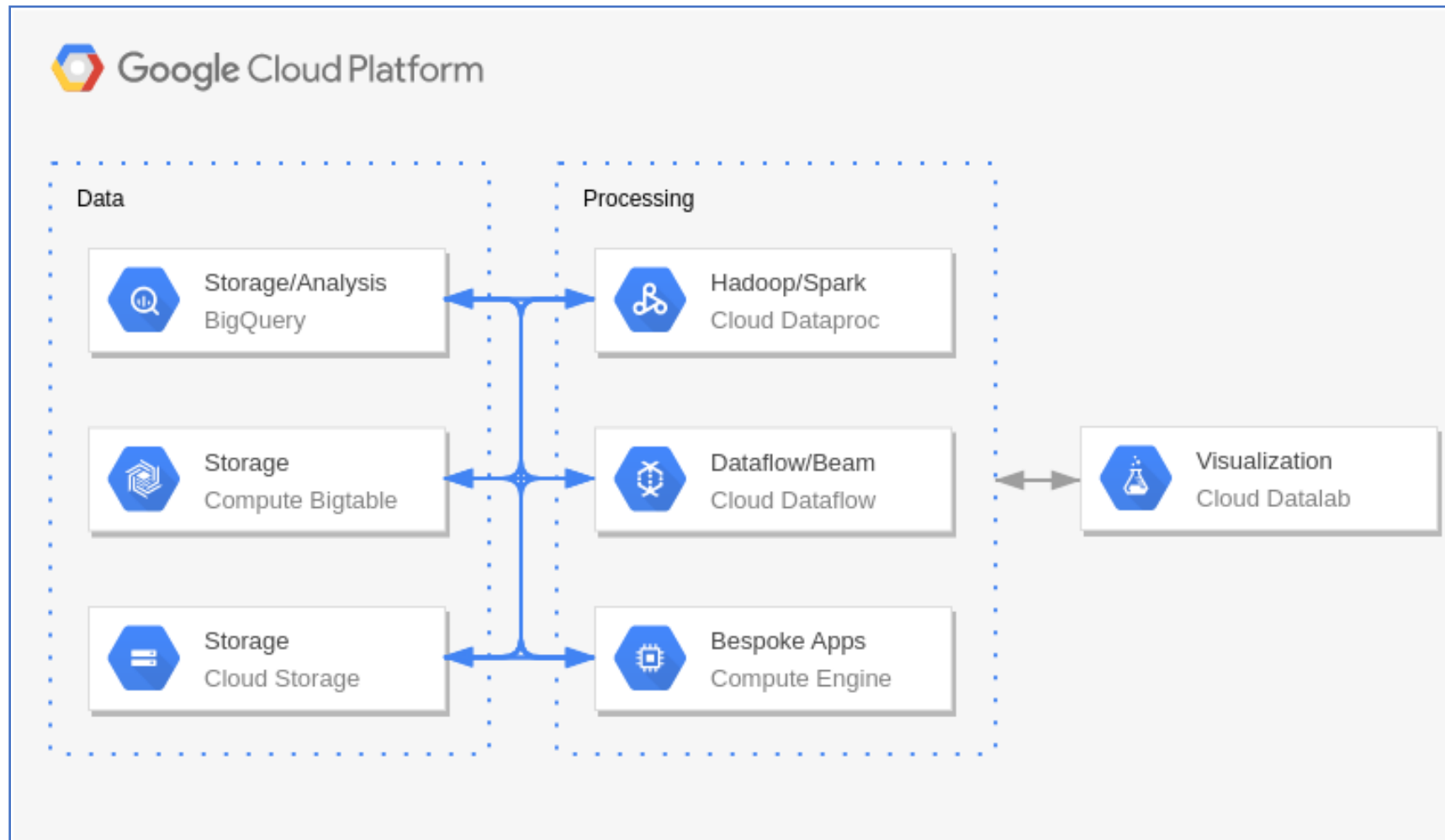
- состояний
- деятельности
- взаимодействия/последовательности/коммуникации

В случае необходимости элементы UML могут быть расширены и переопределены !

Облачные сервисы

<https://online.visual-paradigm.com>

<https://www.smartdraw.com/uml-diagram>



Прагматика применения диаграмм

- ✓ Простые идеи и нотация
 - Применяется на всех фазах жизненного цикла
 - Понимают все (разработчики, заказчики, управленцы) одинаково
- ✓ Моделирование использования позволяет начинающему разработчику совершать меньше грубых ошибок на ранних этапах проектирования
- ❖ Опытный разработчик может использовать другие подходы

диаграммы моделирования поведения

Диаграммы состояний

Диаграммы деятельности

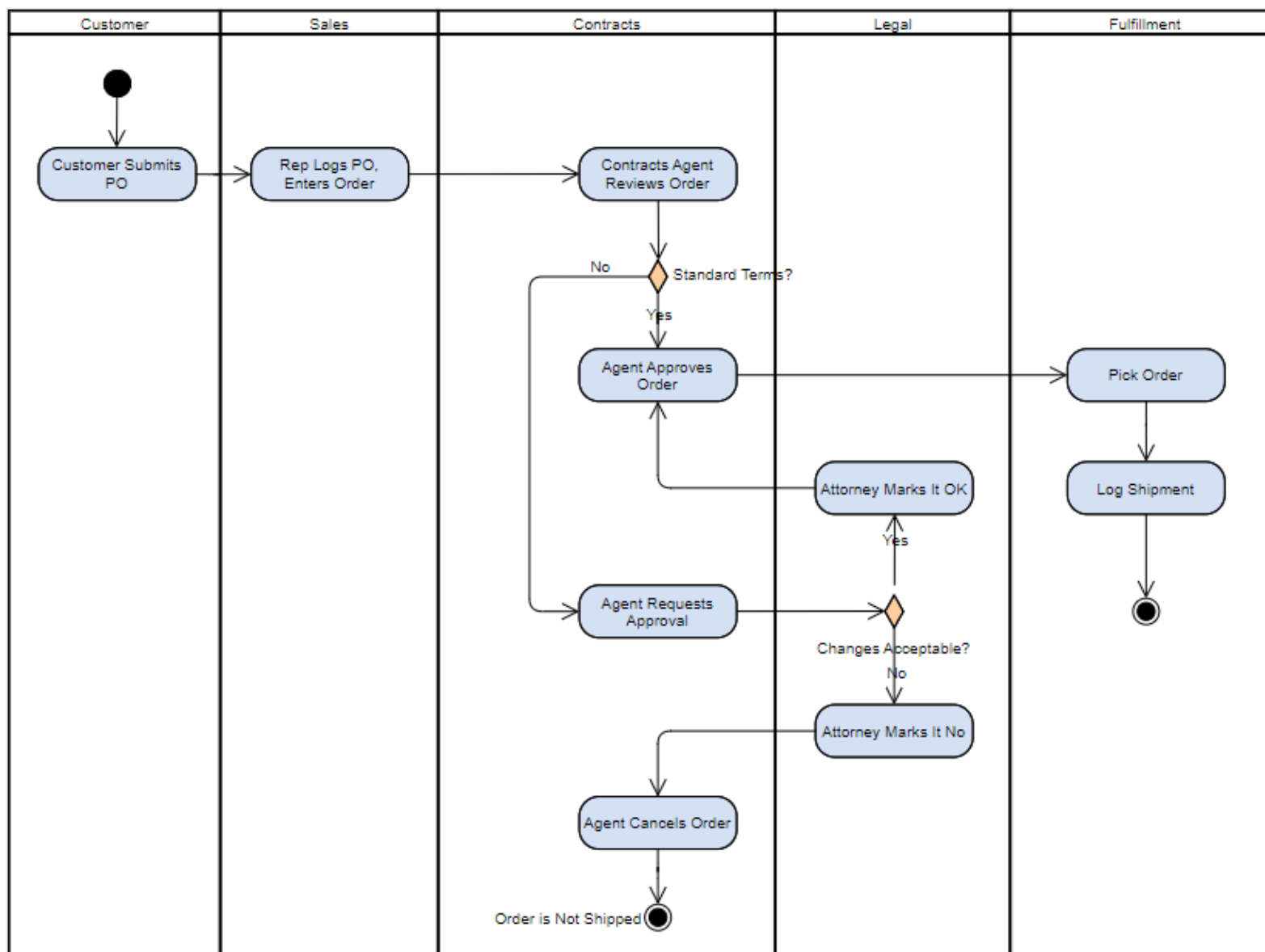
Диаграммы взаимодействия

Диаграммы коммуникации

Диаграммы последовательности

Моделирование параллелизма

Диаграмма деятельности



Диаграммы деятельности

Диаграммы деятельности позволяют моделировать сложный жизненный цикл объекта, с переходами из одного состояния (деятельности) в другое.

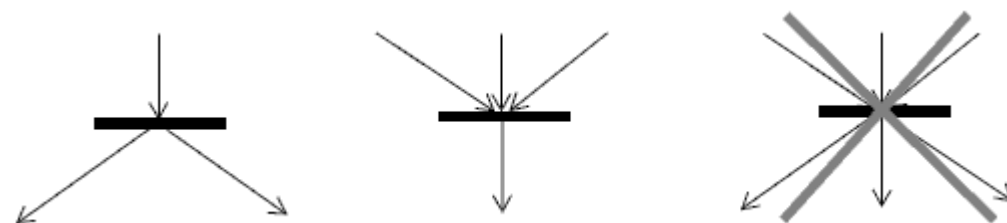
Этот вид диаграмм может быть использован для описания динамики совокупности объектов. Они применимы для детализации некоторой конкретной операции. Предоставляют больше возможностей, чем классические блок-схемы.

Диаграммы деятельности описывают переход от одной деятельности к другой, в отличие от диаграмм взаимодействия, где акцент делается на переходах потока управления от объекта к объекту.

Деятельность - это протяженное по времени составное действие.

Диаграммы деятельности

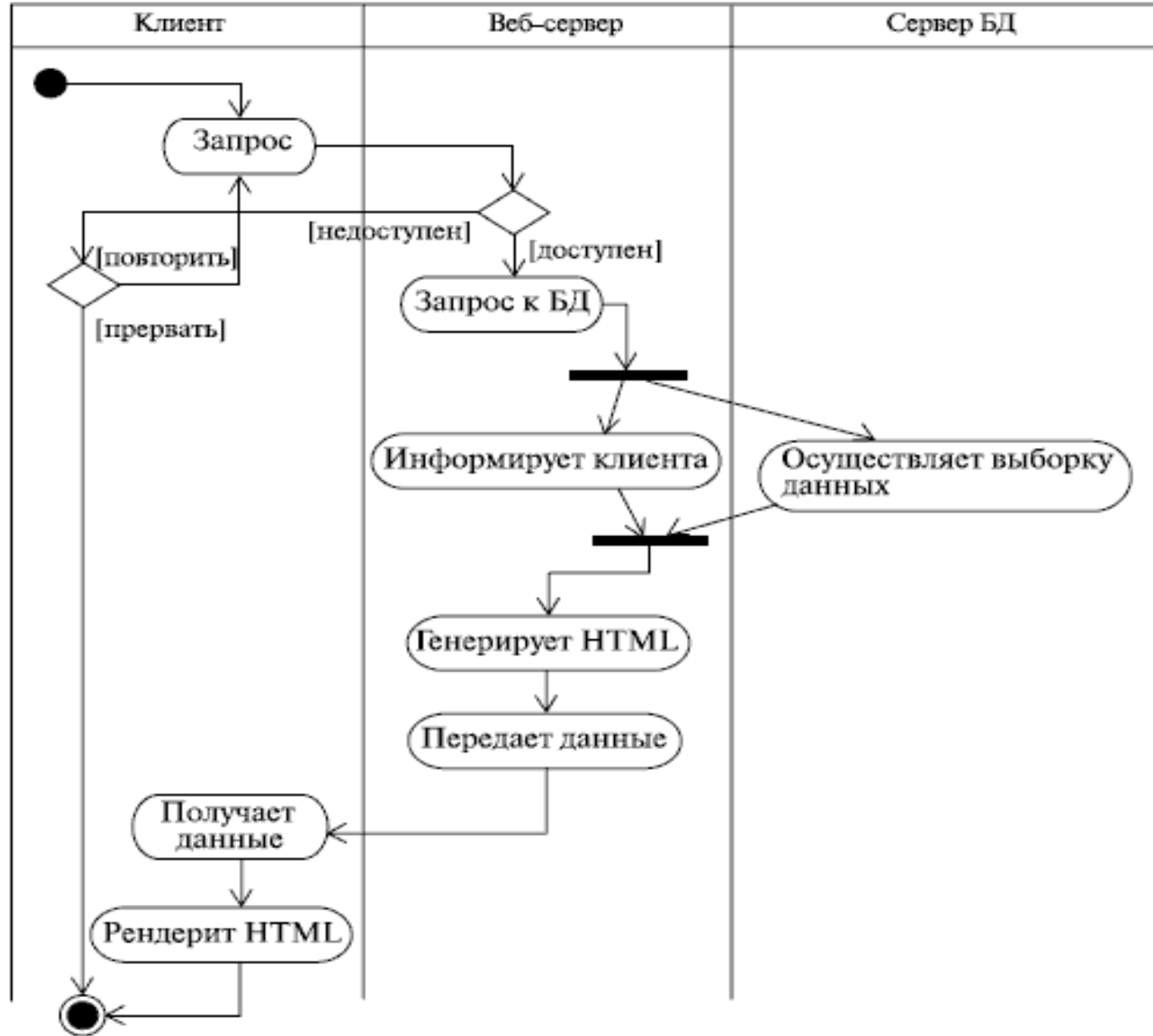
- Деятельность (activity)
- Действие (action)
- Переходы (transitions)
- Ветвления (decisions)
- Развилки и слияния (fork & join)
- Дорожки (swim lanes)
- Траектория объекта (object flow)
- Посылка и прием сигналов
- Прерывания и исключения



Отображение *распараллеливания*, а затем слияние воедино (*синхронизация*) потоков управления, т.е. *операции* выполняются *одновременно*.

Нотация проста: несколько потоков управления сливаются в один или один *поток* разделяется на несколько.

Диаграммы деятельности



Применение диаграмм деятельности

«Создавая диаграммы деятельности, не забывайте, что они моделируют лишь срез некоторых динамических аспектов поведения системы.

С помощью единственной диаграммы деятельности никогда не удастся охватить все динамические аспекты системы.

Вместо этого следует использовать разные диаграммы деятельности для моделирования динамики рабочих процессов и отдельных операций»

Грэди Буч /разработчик UML/

Применение диаграмм деятельности

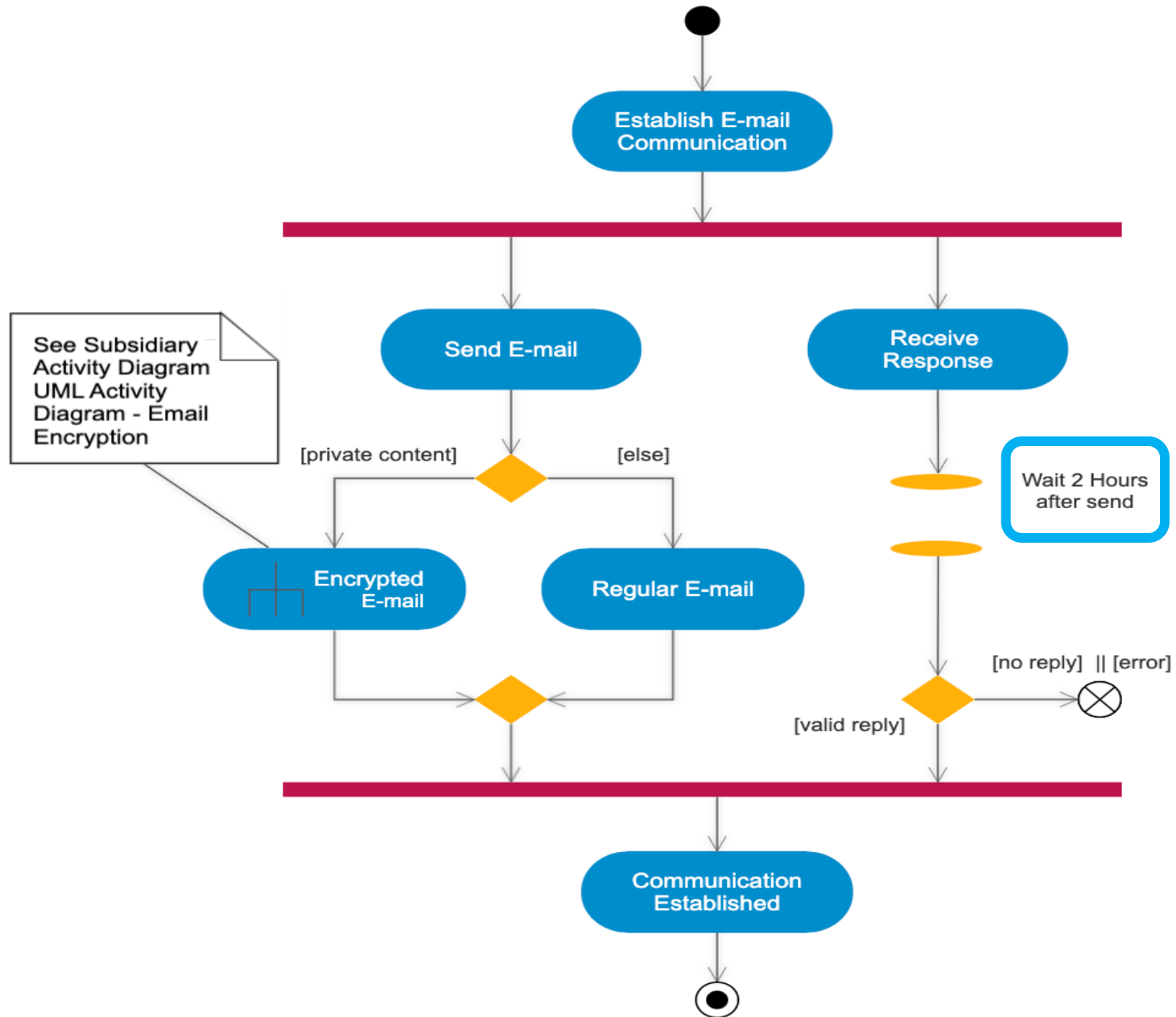
Для моделирования процессов

- Важны для деятельности с точки зрения акторов, работающих с системой, часто при описании бизнес-процессов. В случае такого использования диаграмм деятельности активно используются траектории объектов.

Для моделирования операций

- Диаграммы деятельности играют роль "продвинутых" блок-схем и применяются для подробного моделирования вычислений.
- При таком использовании важны конструкции принятия решения и разделения/слияния потоков управления (синхронизации).

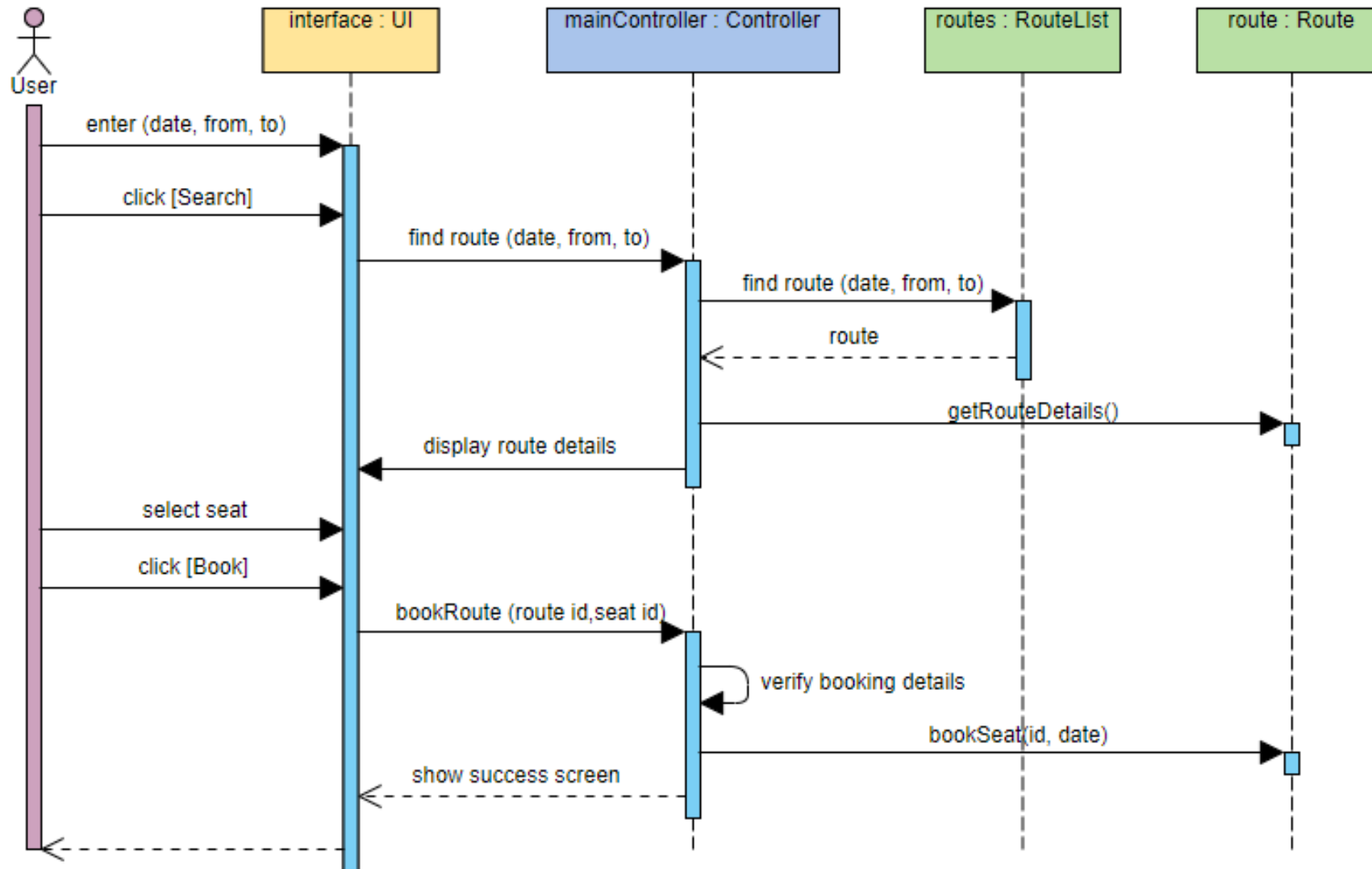
Диаграммы деятельности



Применение диаграмм деятельности

- ✓ Диаграммы деятельности являются частным случаем диаграммы состояний.
- ✓ Диаграммы деятельности могут отображать одновременно выполняемые действия.
- ✓ На диаграммах активности можно использовать дорожки, распределяющие деятельности в соответствии с ролями (объектами), их выполняющими.
- ✓ Траектория объекта позволяет показать объекты, относящиеся к деятельности, и моменты переходов этих объектов из одного состояния в другое.
- ✓ Сложные деятельности можно дополнительно детализировать, разбив на действия и изобразив "диаграмму в диаграмме".
- ✓ Диаграммы деятельности можно использовать для проектирования процессов (бизнес-процессов) или операций (вычислений).

Диаграмма последовательности



Применение диаграмм взаимодействия

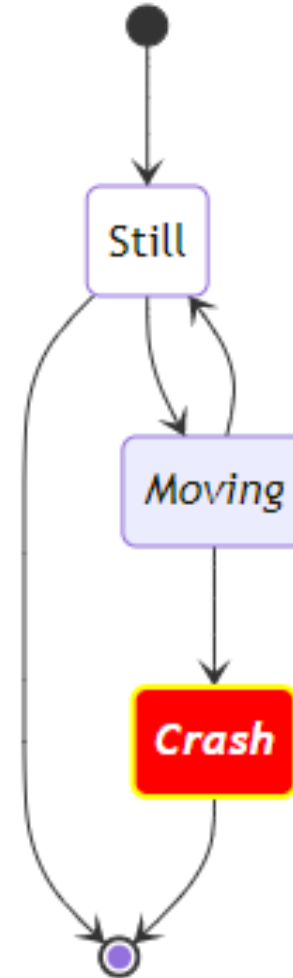
- ✓ Временная диаграмма — UML-диаграмма взаимодействия, которая фокусируется на процессах, происходящих в течение определенного периода времени.
- ✓ Особый случай диаграммы последовательности, в которой время увеличивается слева направо.

Применение метода «конечных автоматов»

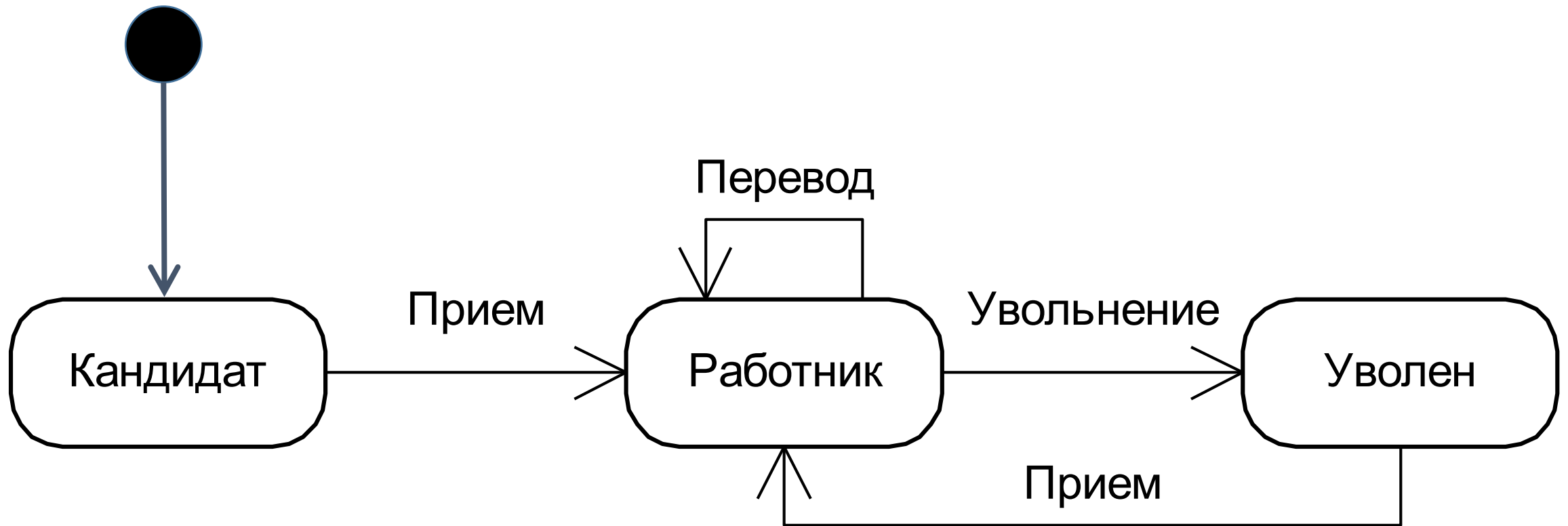
- Трансляция / интерпретация // компиляторы
- Пользовательский интерфейс
- Программы, управляемые событиями
- Моделирование жизненного цикла объекта
 - Если поведение зависит от прошлого
 - Если есть асинхронные стимулы

Применение диаграммы состояний

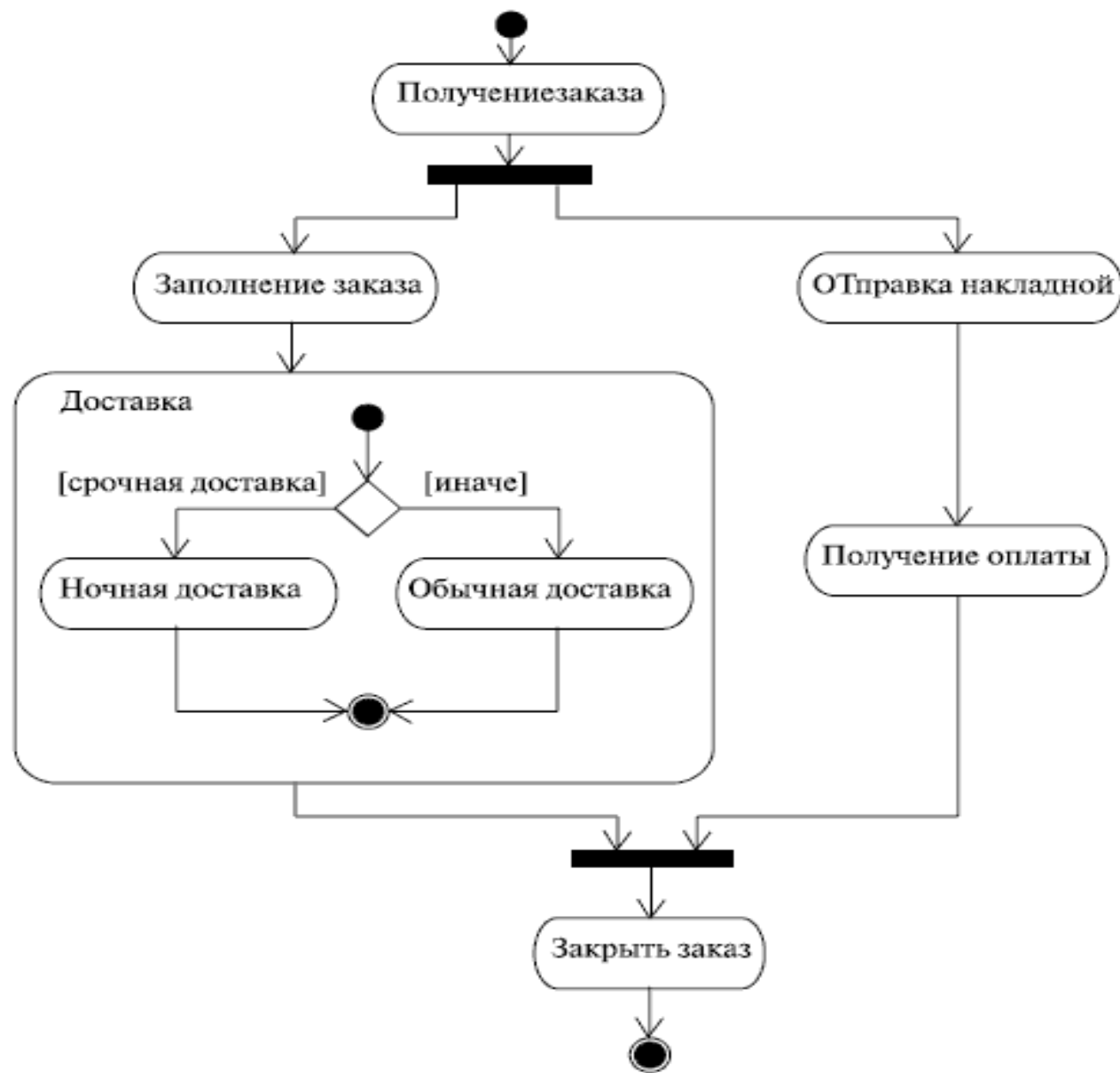
- Состояния (state)
 - Простые (simple)
 - Составные (composite)
 - Специальные (pseudo)
- Переходы (transition)
 - Начало (source)
 - Событие (event)
 - Условие (guard condition)
 - Действие (action)
 - Конец (target)



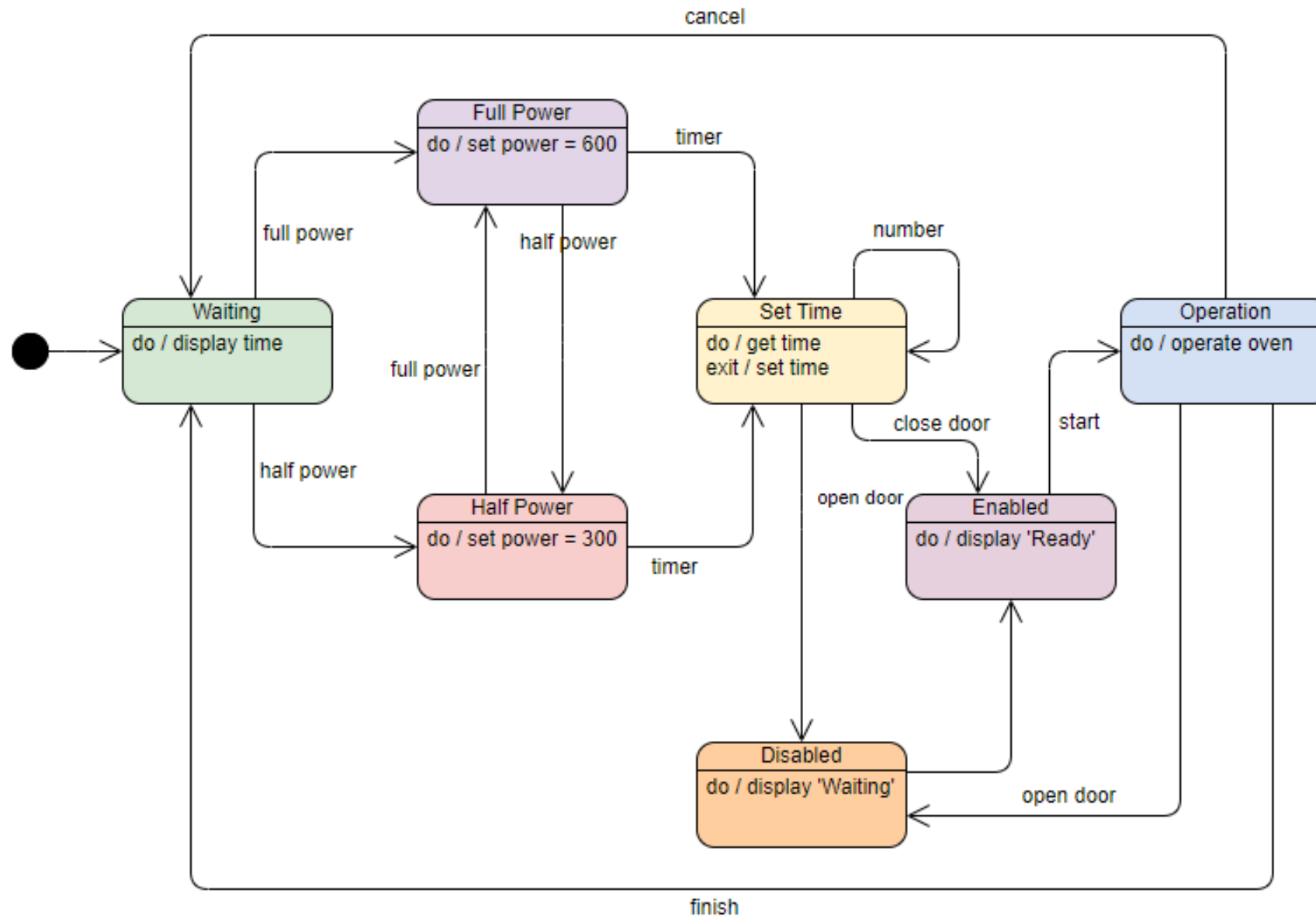
Пример диаграммы состояний сотрудника



Пример диаграммы состояний заказа



Пример диаграммы состояний СВЧ



Сложные диаграммы состояний

Простое состояние — это состояние, не имеющее подструктуры.

Состояние, которое имеет под-состояния (вложенные состояния), называется **составным состоянием**.

Под-состояния могут быть вложены на любом уровне. Вложенный конечный автомат может иметь не более одного начального и одного конечного состояния.

Под-состояния используются **для упрощения сложных** плоских автоматов состояний, показывая, что некоторые состояния возможны только в определенном контексте (окружающее состояние).

Связанные состояния могут быть сгруппированы в одно **составное состояние**. Вложение состояний внутрь других необходимо, если действие включает **параллельные** под-действия.

Mermaid - Инструментарий для схем описаний

<https://mermaid.js.org/>

Mermaid.js интегрирована в GitLab, GitHub.

Есть плагины для множества других сервисов.

Для редакторов кода VSCode, IDEA есть дополнения, которые помогают рендерить диаграммы.

Mermaid помогает быстрее и эффективнее оформлять схемы и графики для документации в виде описаний генерации диаграмм.

Блок-схемы состоят из узлов в виде геометрических фигур и стрелок, соединяющих узлы. Mermaid позволяет создавать динамические блок-схемы. Блок-схема создается с помощью ключевого слова **flowchart** и аббревиатуры для указания направления (**TD, LR**).

В узел можно поместить любой текст, если после имени указать текст в квадратных скобках. Форма узла задается символами вокруг текста.

Mermaid - Инструментарий для схем описаний

flowchart LR

node1[Форма 1]

node2(Форма 2)

node3([Форма 3])

node4[[Форма 4]]

node5[(Форма 5)]

node6((Форма 6))

node14((((форма 14))))

flowchart LR

node7>Форма 7

node8{Форма 8}

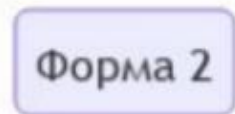
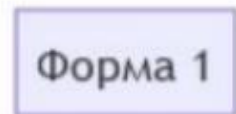
node9{{Форма 9}}

node10[/Форма 10/]

node11[\Форма 11\]

node12[/Форма 12\]

node13[\Форма 13/]



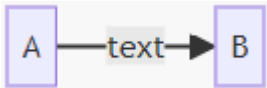
Mermaid - Инструментарий для схем описаний

Узлы в блок-схеме соединяются с помощью стрелок и линий.

A --> B



A -- text --> B



A <--> B



A -.-x B



A x- - -x B



A -.-o B

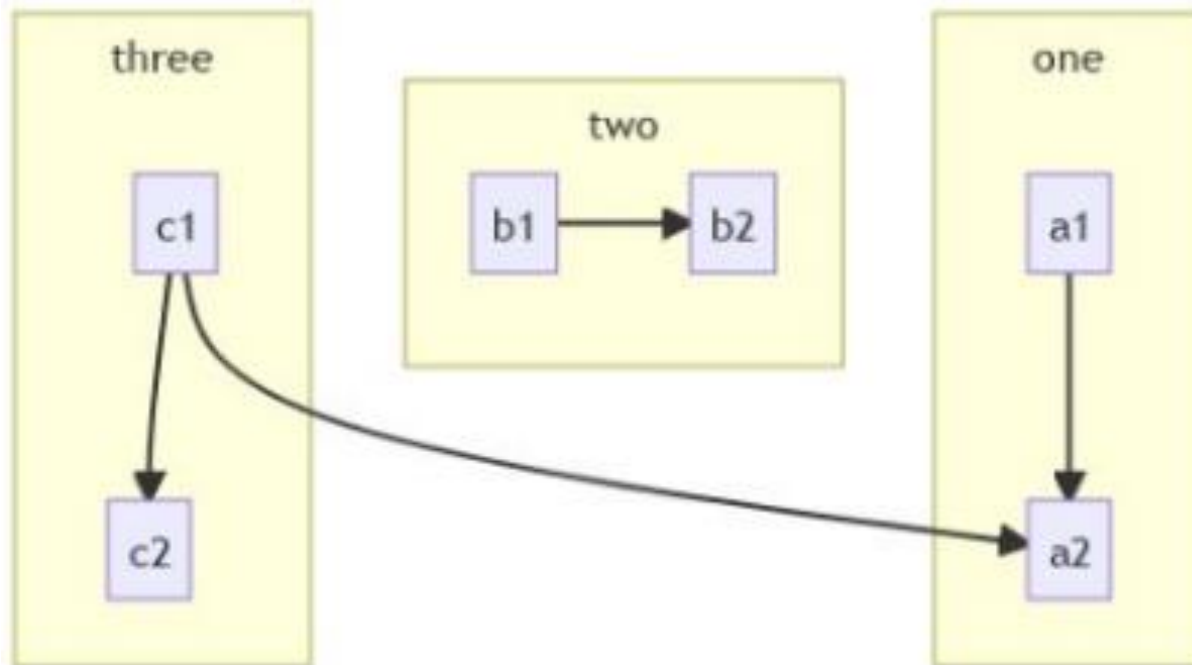


Можно размещать текст на стрелках и связях узлов.

	длина1	длина2
Нормальная	- - -	- - - -
Невидимая	~ ~ ~	~ ~ ~ ~
Нормальная стрелка	- - ->	- - - ->
Толстая	= = =	= = = =
Толстая стрелка	= = =>	= = = =>
Пунктирная	- . -	- . . . -
Пунктирная стрелка	- . ->	- . . . ->
2х-сторонняя толстая стрелка	< = = = >	< = = = = >

Mermaid - Инструментарий для схем описаний

Если на одной схеме необходимо показать взаимосвязанную работу двух алгоритмов, то для этих целей подойдут подсхемы, позволяющие визуально разделять несколько сущностей на одном рисунке.



flowchart TD

c1--> a2

subgraph one

a1--> a2

end

subgraph two

b1--> b2

end

subgraph three

c1--> c2

end

Mermaid - Инструментарий для схем описаний

Узлам можно задавать альтернативные цвета. При этом для каждого отдельно взятого узла можно задать свой цвет.

Для этого после описания блок-схемы пишется ключевое слово **style**, после указать имя узла, потом описать цветовую схему. Цветовая схема задается с помощью тегов:

fill - заливка; *stroke* - цвет границы; *stroke-width* - толщина границы;
color - цвет текста; *stroke-dasharray* - пунктирная граница.

flowchart LR

id1(Start) --> id2(Stop)

style id1 fill:green, stroke:#333, stroke-width:4px;

style id2 fill:red, stroke:#333, stroke-width:4px, color:#fff, stroke-dasharray: 12 5;

classDef class1 fill:green, stroke:#333, stroke-width:4px;

classDef class2 fill:red, stroke:#333, stroke-width:4px, color:#fff, stroke-dasharray: 12 5;



Mermaid - Инструментарий для схем описаний

UML-диаграммы используются для схематичного отображения классов и их связей в коде. Здесь можно создавать диаграммы классов с помощью ключевого слова `classDiagram`. Каждый экземпляр класса содержит в себе три составляющие: в верхней части располагается название класса и описание; в центральной части находятся атрибуты класса, которые указываются со строчной буквы и выравниваются по левому краю; в нижней части размещены методы класса.

```
classDiagram
class BankAccount
BankAccount : +String owner
BankAccount : +BigDecimal balance
BankAccount : +deposit(amount)
BankAccount : +withdrawl(amount)
```

Или

```
classDiagram
class BankAccount {
+String owner
+BigDecimal balance
+deposit(amount)
+withdrawl(amount) }
```



Mermaid - Инструментарий для схем описаний

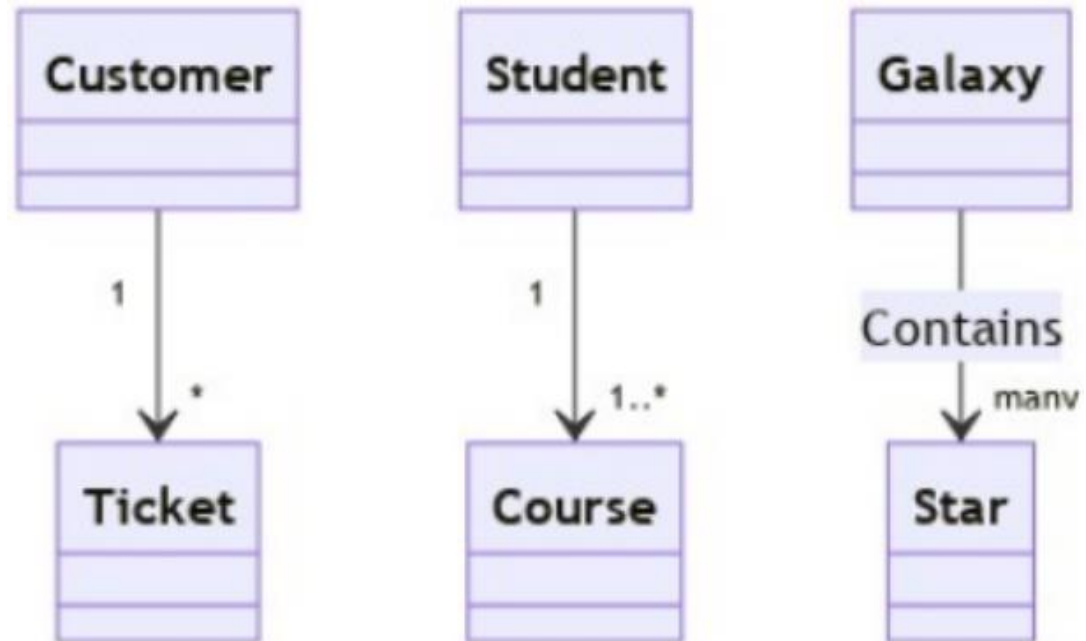
Отношения между классами задаются с помощью различных видов стрелок.

При желании можно явно указать вид связи в виде текста.

Двусторонние отношения задаются с помощью дублированной стрелки, направленной в обе стороны. Двусторонние отношения возможны для следующих видов взаимоотношений:

< Наследование	* Композиция	o Агрегация
> Ассоциация	< Ассоциация	> Реализация

```
classDiagram
  Customer "1" --> "*" Ticket
  Student "1" --> "1..*" Course
  Galaxy --> "many" Star: Contains
```



Mermaid - Инструментарий для схем описаний

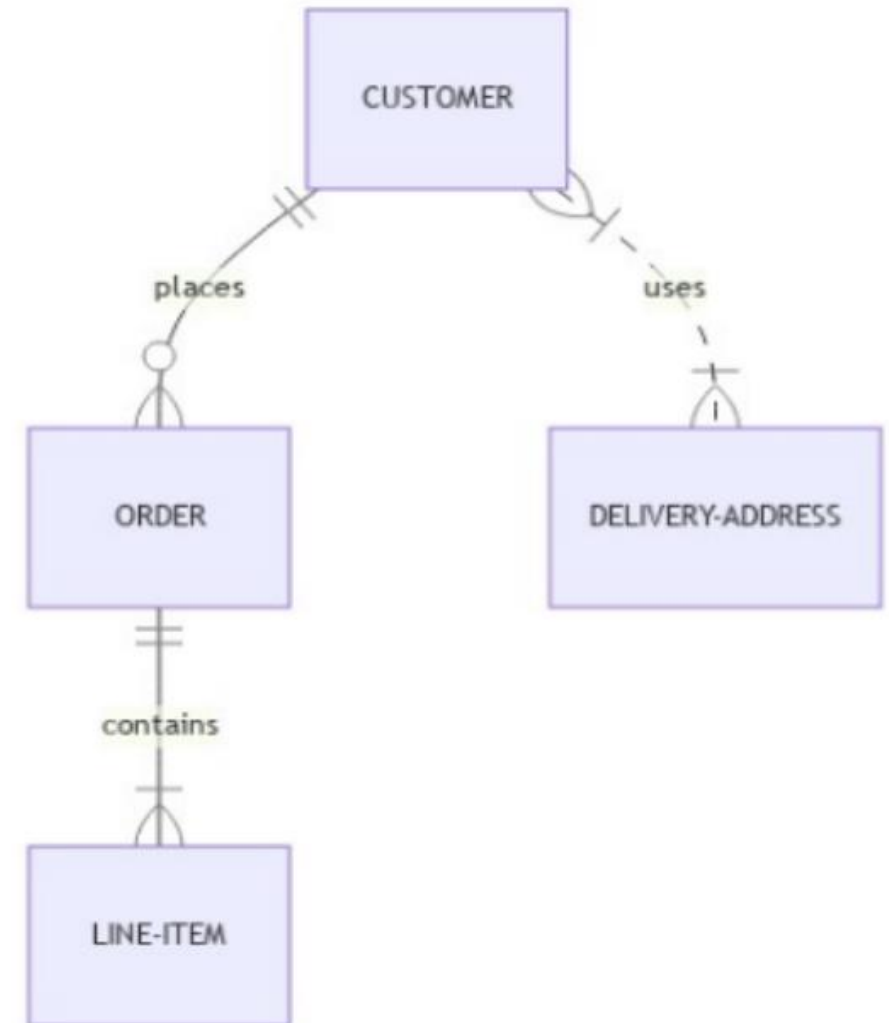
Модель данных, описывающая взаимосвязанные вещи, представляющие интерес в определенной области знаний, используется при высокоуровневом проектировании баз данных. Задается с помощью ключевого слова *erDiagram*

erDiagram

```
CUSTOMER ||--o{ ORDER : places
```

```
ORDER ||--}|{ LINE-ITEM : contains
```

```
CUSTOMER }|..}|{ DELIVERY-ADDRESS : uses
```



Mermaid - *Инструментарий для схем описаний*

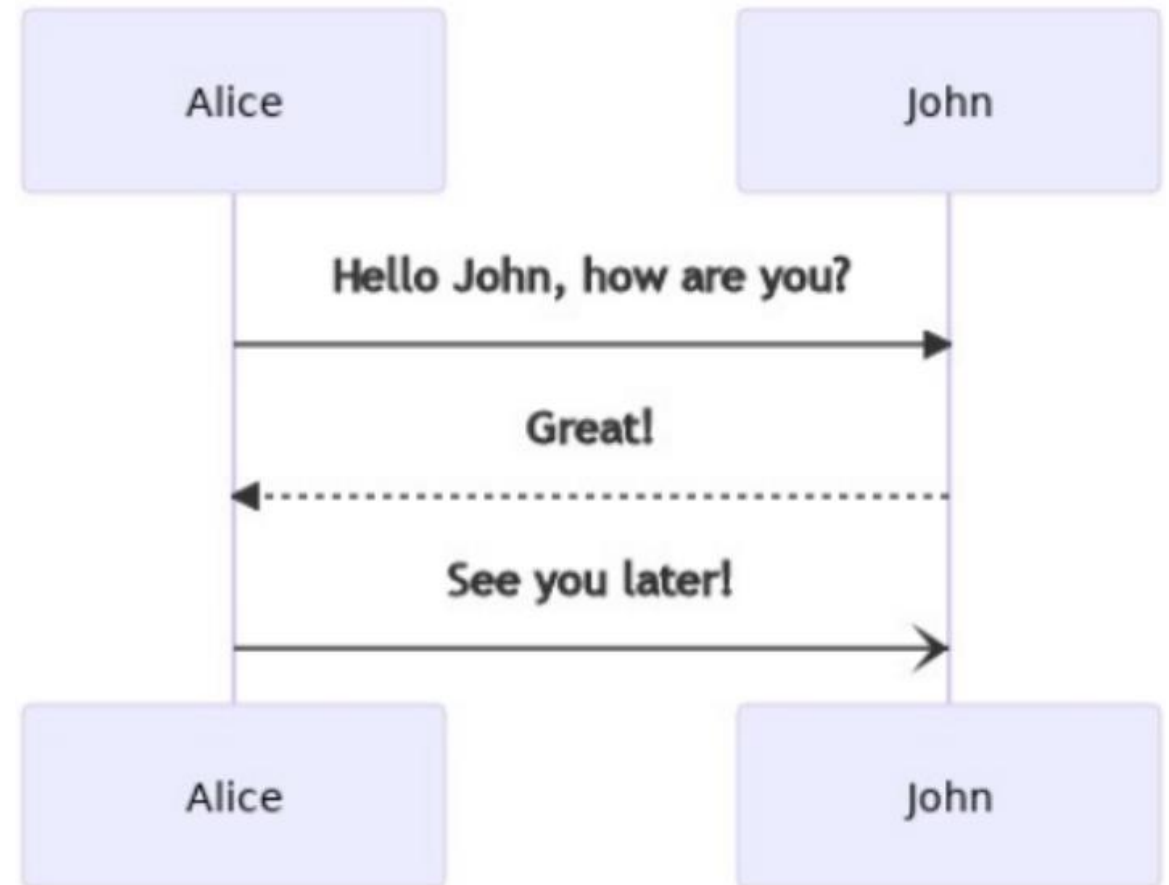
На диаграммах последовательности отображается жизненный цикл одного объекта или нескольких, а также их взаимодействие. Диаграммы последовательности состоят из акторов в виде прямоугольников и взаимодействий в виде вертикальных «линий жизни». В Mermaid диаграммы последовательности задаются с помощью ключевого слова `sequenceDiagram`

`sequenceDiagram`

`Alice->>John: Hello John, how are you?`

`John-->>Alice: Great!`

`Alice-)John: See you later!`



Mermaid - Инструментарий для схем описаний

gitGraph LR:

commit id: "1"

commit id: "2"

branch nice_feature

checkout nice_feature

commit id: "3"

checkout main

commit id: "4"

checkout nice_feature

branch very_nice_feature

checkout very_nice_feature

commit id: "5"

checkout main

commit id: "6"

checkout nice_feature

commit id: "7"

checkout main

merge nice_feature id: "customID" tag: "customTag" type: REVERSE

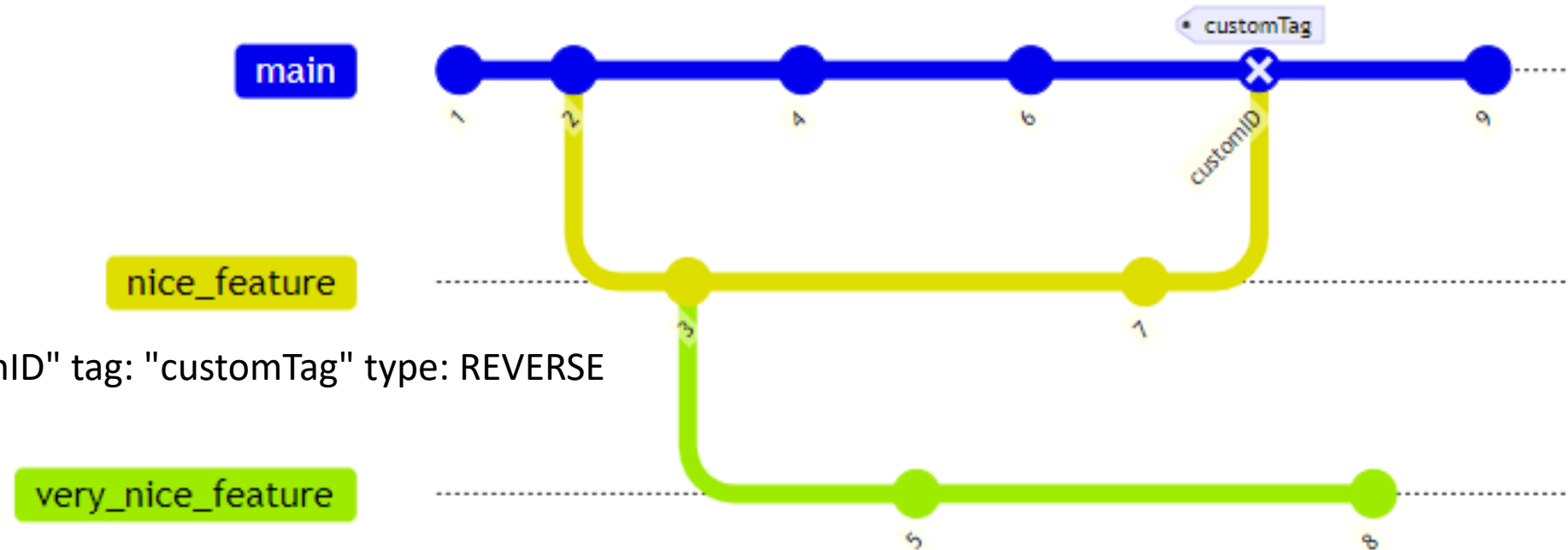
checkout very_nice_feature

commit id: "8"

checkout main

commit id: "9"

Mermaid может отображать диаграммы Git. Подобные диаграммы полезны командам разработчиков, поскольку они позволяют поделиться своими вариантами ветвления Git. Это упрощает визуализацию того, как работает поток git.



Mermaid - Инструментарий для схем описаний

gantt

dateFormat YYYY-MM-DD

title Adding GANTT diagram functionality

section A section

Completed task :done, des1, 2022-01-06,2022-01-08

Active task :active, des2, 2022-01-09, 3d

Future task : des3, after des2, 5d

Future task2 : des4, after des3, 5d

section Critical tasks

Completed task in the critical line :crit, done, 2022-01-06,24h

Implement parser and json :crit, done, after des1, 2d

Create tests for parser :crit, active, 3d

Future task in critical line :crit, 5d

Create tests for renderer :2d

Add to mermaid :until isadded

Functionality added :milestone, isadded, 2022-01-25, 0d

section Documentation

Describe gantt syntax :active, a1, after des1, 3d

Add gantt diagram to demo page :after a1 , 20h

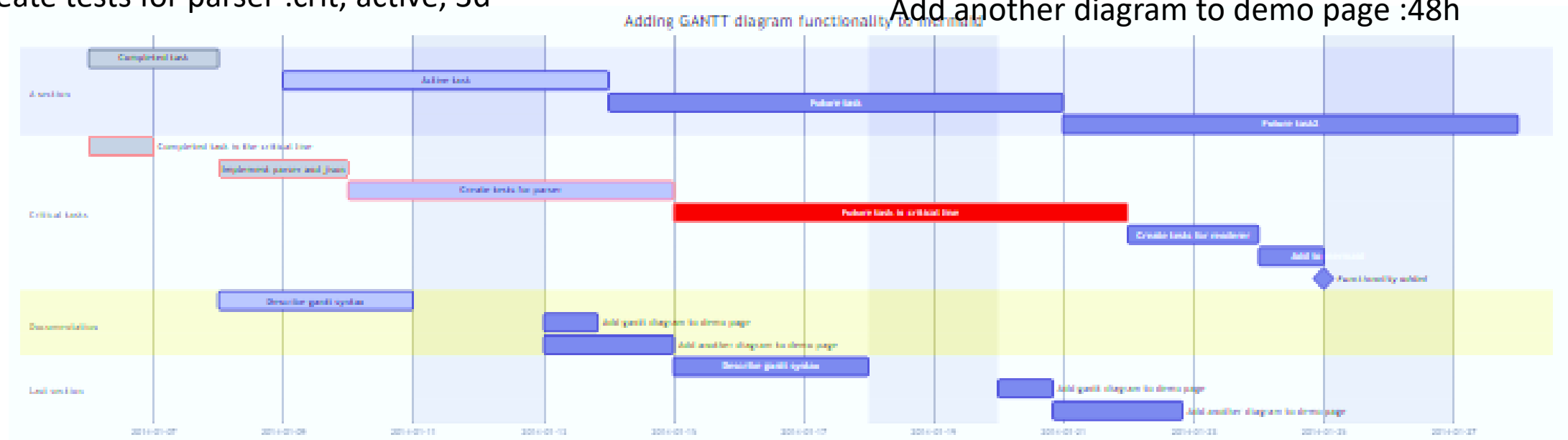
Add another diagram to demo page :doc1, after a1 , 48h

section Last section

Describe gantt syntax :after doc1, 3d

Add gantt diagram to demo page :20h

Add another diagram to demo page :48h



Mermaid - Инструментарий для схем описаний

Узел можно сделать кнопкой, открывающей страницу в браузере. Для этого надо указать ссылку в описании узла. Ссылку необходимо экранировать кавычками.

```
flowchart TD
```

```
%% создаем узел
```

```
  A --> B
```

```
%% Комментарии задаются двумя знаками процента
```

```
%% Описываем действия для клика по узлу
```

```
click A "https://www.bmstu.ru" _blank
```

Можно настроить инициализацию и умолчания:

```
%%{init: { 'logLevel': 'debug', 'theme': 'default',  
  'themeVariables': { 'commitLabelColor': '#ff0000',  
    'commitLabelBackground': '#00ff00' } } }%%
```

Mermaid – плагины



ChatGPT plugin

The ChatGPT plugin leverages generative AI, providing a seamless transition to Mermaid Chart for editing, and enhances communication with visual clarity and simplified workflow.

[Learn More](#)

[Go to Download](#)



Visual Studio Code plugin

The Mermaid Chart plugin for Visual Studio Code (a source-code editor made by Microsoft) seamlessly integrates Mermaid Chart into your coding workflow.

[Learn More](#)

[Go to Download](#)



JetBrains IDE extension

The Mermaid Chart extension for JetBrains IDE extension JetBrains IDEs enables developers to view and edit diagrams stored in Mermaid Chart within the IDE.

Copyright © 2000–2023 JetBrains s.r.o. JetBrains and the JetBrains logo are registered trademarks of JetBrains s.r.o.

[Learn More](#)

[Go to Download](#)



Microsoft PowerPoint plugin

The Mermaid Chart plugin for Microsoft PowerPoint presentation processing program seamlessly integrates Mermaid Chart into your workflow.

[Learn More](#)

[Go to Download](#)



Microsoft Word plugin

The Mermaid Chart plugin for Microsoft PowerPoint and Word (a word processing program made by Microsoft) seamlessly integrates Mermaid Chart into your workflow.

[Learn More](#)

[Go to Download](#)

Mermaid - графы

Процесс приготовления кофе:

flowchart LR

A[Start]-->B[Do you have coffee beans?]

B-->|Yes|C[Grind the coffee beans]

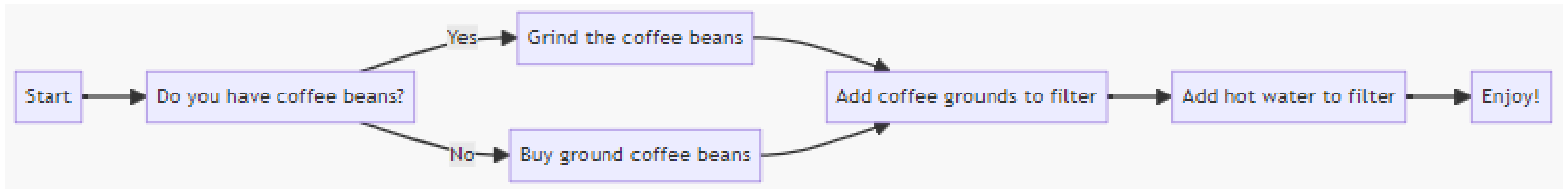
B-->|No|D[Buy ground coffee beans]

C-->E[Add coffee grounds to filter]

D-->E

E-->F[Add hot water to filter]

F-->G[Enjoy!]



Mermaid - графы

flowchart LR

Процесс поиска неисправности автомобиля

A[Start: Problem - Car won't start] --> B{Is the battery dead?}

B -- Yes --> C[Replace battery]

B -- No --> D{Is the fuel tank empty?}

D -- Yes --> E[Refill fuel tank]

D -- No --> F{Is the engine oil level low?}

F -- Yes --> G[Refill engine oil]

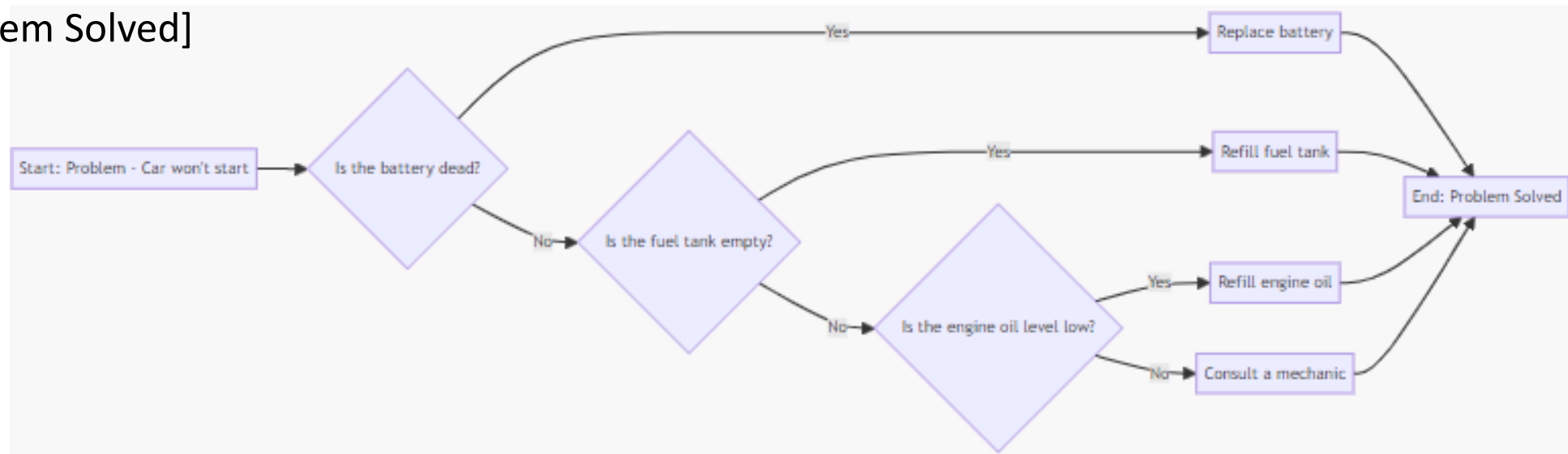
F -- No --> H[Consult a mechanic]

C --> I[End: Problem Solved]

E --> I

G --> I

H --> I



Mermaid - графы

Процесс доставки заказа

flowchart LR

A(Order Received) --> B[Order Processing]

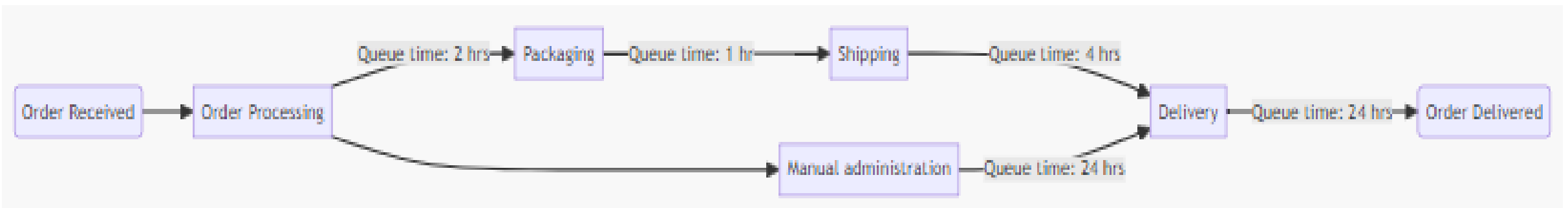
B --> |Queue time: 2 hrs| C[Packaging]

C --> |Queue time: 1 hr| D[Shipping]

D --> |Queue time: 4 hrs| E[Delivery]

E --> |Queue time: 24 hrs| F(Order Delivered)

B --> G[Manual administration]--Queue time: 24 hrs--> E



Плагин для Visual Studio Code

Это расширение – инструмент для визуализации и редактирования диаграмм Mermaid в Visual Studio Code. Он позволяет разработчикам просматривать и редактировать диаграммы, хранящиеся в Mermaid Chart, непосредственно из кода Visual Studio, упрощая рабочий процесс разработки.

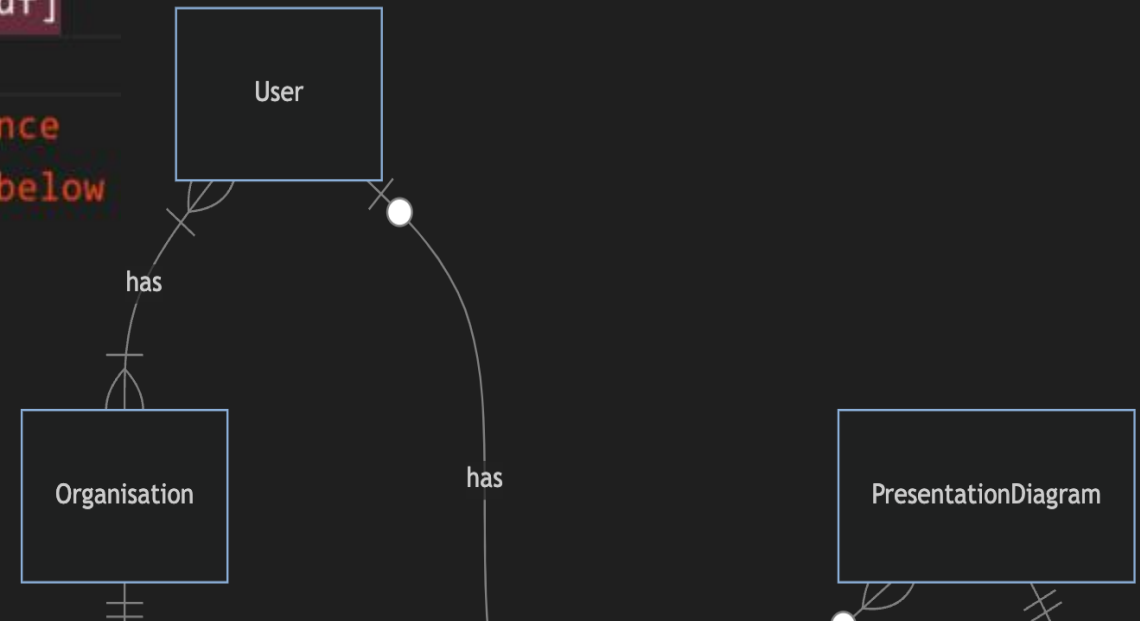
Расширение Mermaid Chart предлагает возможности:

- Прикрепить диаграммы к своему коду в виде значка в нижнем колонтитуле
- С каждой диаграммой связаны две ссылки: одна для просмотра диаграммы на новой вкладке в Visual Studio Code, а другая для редактирования диаграммы в Mermaid Chart

Плагин для Visual Studio Code

```
JS test.js  X  Settings
JS test.js > aFunction
1  /** A function filtering odd number out of an array */
2  function filterOddNumbers(array) {
3      return array.filter((n) => n % 2 !== 0);
4  }
5
View Diagram | Edit Diagram
6  // [MermaidChart: 4d050feb-1f8e-424f-8c4b-dac0ea6620df]
7  function aFunction() {
8      // Test function illustrating some code, for instance
9      // The version of a bubble sort copilot generated below
```

50feb-1f8e-424f-8c4b-dac0ea6620df X



Плагин для JetBrains IDE

Это расширение – инструмент для визуализации и редактирования диаграмм Mermaid в IntelliJ IDEA, Android Studio, AppCode, Aqua, CLion, DataGrip, DataSpell, GoLand, JetBrains Client, JetBrains Gateway, MPS, PhpStorm, PyCharm, Rider, RubyMine, RustRover, WebStorm.

Расширение Mermaid Chart предлагает возможности:

- Прикрепить диаграммы к своему коду в виде значка в нижнем колонтитуле
- С каждой диаграммой связаны две ссылки: одна для просмотра диаграммы на новой вкладке в IntelliJ IDEA, а другая для редактирования диаграммы в Mermaid Chart

Плагин Mermaid Chart для ChatGPT

Есть для чата ChatGPT для создания диаграмм Mermaid.

С помощью простых инструкций, таких как «Create a State diagram showing the states of a stereo» или «Draw a timeline of World War II», можно создавать текстовые диаграммы в ходе чата.

После создания диаграммы ее можно доработать в Mermaid Chart.

Используйте возможности генеративного искусственного интеллекта. Воспользуйтесь обширной базой знаний ChatGPT.

Плагин предоставляет отправную точку для синтаксиса диаграмм, и извлекает соответствующую информацию из Интернета для заполнения содержимого диаграмм без необходимости программирования или дизайна.